# Common Time Complexities

DSA lecture notes, part 2 (from the YouTube lecture, course “Complete DSA from A to Z”). The lecture is in Hinglish; these notes are in simple English with the key terms kept as in the lecture.

**In one minute**

- Most algorithms you meet fall into a handful of time complexities. Learn these and how their graphs look.
- From best to worst: **O(1)** constant, **O(log n)** logarithmic, **O(n)** linear, **O(n log n)** linear-logarithmic, **O($n^2$)** quadratic, **O($2^n$)** exponential, **O(n!)** factorial.
- To get the complexity, count how the number of steps grows as the input size n grows. **Ignore constants and smaller terms**: $an^2 + bn + c$ becomes $O(n^2)$.
- Differences only matter for **large inputs**, so we always look at the **worst case (Big O)**.

## 1. Setup used in the lecture (Java)

- Make a folder for practice (for example **week1**), open it in **VS Code**.
- Install **Java (JDK)** from Oracle for your operating system: an installer for Windows, a DMG for Mac, or an RPM/Debian package for Linux. The lecture used the stable LTS version 21 (23 was just released); a newer stable version is also fine.
- In VS Code install the **Extension Pack for Java**. If the Run button does not show, install the **Code Runner** extension.
- Test with a `Test.java` file containing a `main` method that prints “Hello DSA World”.

Run it with the Run button, or from the terminal:

```
javac Test.java     # compile
java Test           # run (use the class name)
```

## 2. O(1): constant time

The time does **not** change however big the input is. The same single statement runs whether the array has 1 element or 10,000.

**Example:** accessing an element of an array by its index. You know the index, so you reach the element in one step.

```
static void constantTime() {
    int[] arr = {1, 2, 3, 4, 5};
    System.out.println(arr[0]);   // one step, whatever the array size
}
```

- **Graph** (x = input size, y = time): a **flat horizontal line**.
- **Equation:** like y = mx + c with the slope m = 0, so T = c (a constant).
- Written O(1), or O(c). This is the best complexity.

## 3. O(n): linear time

Time grows **in direct proportion** to the input size. Double the input and the time doubles.

**Example:** traversing (visiting every element of) an array of size n. 4 elements means 4 visits, 5 means 5, 8 means 8.

```
static void linearTime() {
    int[] arr = {1, 2, 3, 4, 5, 6, 7, 8};
    for (int i = 0; i < arr.length; i++) {
        System.out.println((i + 1) + " " + arr[i]);   // runs n times
    }
}
```

The lecture shows this by running the code: with 8 elements the print statement runs 8 times; after adding 2 more elements it runs 10 times. Steps grow at the same rate as the input.

- **Graph:** a **straight line going up** to the right.
- **Equation:** y = mx + c, which for time becomes T(n) = an + c. Here n is the input size and a, c are constants.

## 4. O($n^2$): quadratic time

Time grows with the **square** of the input size. This is the “dangerous” one: it is not even linear, and it gets slow quickly.

| **Input size n** | **1** | **2** | **3** | **10** | **100** |
|---|---|---|---|---|---|
| Steps ($n^2$) | 1 | 4 | 9 | 100 | 10,000 |

**Example: bubble sort** (sorts elements from small to large). It checks pairs of elements. There is an outer loop (i) and, **for each i, an inner loop (j)** that goes through the elements again. So the work is n × n.

- In **pass 1** the inner loop compares neighbours and swaps them if they are in the wrong order. At the end the largest element has reached the last position and is left alone.
- In the next pass the same thing happens for the remaining elements, and so on until everything is sorted.

```
for (int i = 0; i < n; i++) {         // outer loop: n times
    for (int j = 0; j < n - 1; j++) {  // inner loop: about n times for each i
        // compare arr[j] and arr[j+1], swap if needed
    }
}
```

- **Graph:** a curve that starts slowly and then rises steeply (a parabola).
- **Equation:** T(n) = $an^2$ + bn + c. For large n the bn and c parts are tiny compared to $an^2$, and constants are ignored, so we write **O($n^2$)**.

## 5. O(log n): logarithmic time

Each step **cuts the problem size in half**. For 100 elements: 100, then 50, then 25, then about 12, and so on, until 1 is left.

**Example: binary search** in a **sorted** array. Look at the middle element. If it is not the one you want, check whether your element is smaller or larger. If smaller, ignore the right half (all larger); if larger, ignore the left half. Repeat on the half that is left.

**Why log n?** After k steps the size is n / $2^k$. We stop when this reaches 1:

| | | | | |
|---|---|---|---|---|
| $n / 2^k = 1$ | so | $n = 2^k$ | so | $k = \log_2 n$ |

So the number of steps is about log n, written **O(log n)**. This is much faster than linear: even for a million elements it takes only about 20 steps.

## 6. O(n log n): linear-logarithmic time

The algorithm **divides** the problem into smaller sub-problems (this gives the log n part) and then **processes each level linearly** (this gives the n part).

- **Examples:** merge sort (divide the array, sort the parts, then merge them) and quick sort.
- Divide: the problem is split in halves again and again, which gives about log n levels.
- Process: at every level all n elements are handled once, which gives n work per level. Total: n × log n.

> **Note:** quick sort is O(n log n) on average, but its worst case is $O(n^2)$.

## 7. $O(2^n)$: exponential time

The time **doubles** with every extra input element. It grows extremely fast.

**Example: recursive Fibonacci.** f(n) is calculated from f(n−1) and f(n−2). Each call splits into two sub-calls, and each of those splits again, so the number of calls blows up like a tree that keeps doubling. It is not even quadratic. The graph shoots almost straight up.

> **Note:** for the plain recursive Fibonacci the exact growth is about $1.6^n$, but it is bounded by $2^n$, which is why it is usually written as $O(2^n)$.

## 8. O(n!): factorial time

Appears in **brute-force** solutions that try **all possible orderings (permutations)**.

- **Example:** the brute-force solution of the **Travelling Salesman Problem**: check every possible order of visiting the cities.
- **Lecture example:** you forgot a 4-digit PIN and try every combination one by one, changing the last digit, then the one before it, and so on, until you hit the right one. This shows the idea of “try everything”.

n! = n × (n−1) × (n−2) × … × 1. It grows even faster than $2^n$, so such algorithms are only usable for very small n.

## 9. Comparing all complexities

From the **best** (smallest time) to the **worst** (largest time):

| Complexity | Name | Example | Rough steps, n = 10 | Rough steps, n = 20 |
|---|---|---|---|---|
| O(1) | Constant | Access array element by index | 1 | 1 |
| O(log n) | Logarithmic | Binary search | about 3 | about 4 |
| O(n) | Linear | Traverse an array | 10 | 20 |
| O(n log n) | Linear-logarithmic | Merge sort | about 33 | about 86 |
| $O(n^2)$ | Quadratic | Bubble sort | 100 | 400 |
| $O(n^3)$, $O(n^k)$ | Polynomial | Three or more nested loops | 1,000 | 8,000 |
| $O(2^n)$ | Exponential | Recursive Fibonacci | 1,024 | about 1 million |
| O(n!) | Factorial | Try all permutations (travelling salesman) | about 3.6 million | about $2.4 \times 10^{18}$ |

The “rough steps” columns are my additions to make the growth easy to see; they are not from the lecture.

Graph, from the lowest curve to the highest: a flat line (constant), a very slowly rising curve (log n), a straight line (linear), a slightly steeper curve (n log n), a parabola ($n^2$), then $n^3$, $n^k$, and finally $2^n$ and n! which shoot up almost vertically.

## 10. Important points from the lecture

- **Ignore constants and smaller terms.** $an^2 + bn + c$ is $O(n^2)$. For big n the small terms hardly matter.
- **Small input sizes hide the difference.** For tiny inputs several complexities can look the same. It matters only when the input is large.
- That is why we **always look at the worst case, Big O**.
- If someone asks “which algorithm is better?”, **find the complexity of each one and compare**: the one with the smaller complexity is better. This is often asked in interviews.
- Next lectures: how to **calculate** the complexity of an algorithm, then data structures, mixed with algorithms.

## 11. Likely interview questions

**Q. Which time complexity is the best and which is the worst among the common ones?**

Best: O(1). Then O(log n), O(n), O(n log n), $O(n^2)$, $O(2^n)$, and O(n!) which is the worst.

**Q. Give an example of O(1).**

Accessing an array element by its index.

**Q. Give an example of O(n).**

Traversing an array once, or linear search in the worst case.

**Q. Give an example of $O(n^2)$.**

Bubble sort, or any pair of nested loops that each run n times.

**Q. Why is binary search O(log n)?**

It halves the remaining input at every step. $n / 2^k = 1$ gives $k = \log_2 n$. The array must be sorted.

**Q. Give examples of O(n log n).**

Merge sort (and quick sort on average): divide into sub-problems, then process them linearly.

**Q. Why is recursive Fibonacci exponential?**

Each call makes two more calls, so the number of calls roughly doubles with every extra level.

**Q. Why do we ignore constants and lower-order terms?**

For large inputs the highest-order term dominates, so the others hardly change the growth rate.

**Q. Why do we use the worst case (Big O)?**

It gives a guarantee that the algorithm will not take longer than this, and the differences between algorithms show up mainly on large inputs.

### A note on the PIN example

> Trying every combination of a 4-digit PIN is brute force, but strictly the number of combinations is $10^4 = 10{,}000$, which is exponential in the number of digits rather than factorial. The lecture uses it to explain “try everything”; the true factorial case is trying all *orderings* of n items, as in the Travelling Salesman Problem.